

Typing the harness: an FTG+PM model of an LLM-assisted development loop and knowledge debt as a conformance property

Vasco Amaral
Universidade Nova de Lisboa
Lisbon, Portugal
alfonso.pierantonio@univaq.it

Miguel Goulão
Universidade Nova de Lisboa
Lisbon, Portugal
alfonso.pierantonio@univaq.it

Ha Phan
Toronto Metropolitan University
Toronto, Canada
alfonso.pierantonio@univaq.it

Alfonso Pierantonio
University of L'Aquila
L'Aquila, Italy
alfonso.pierantonio@univaq.it

Manuel Wimmer
University of L'Aquila
L'Aquila, Italy
alfonso.pierantonio@univaq.it

ABSTRACT

Large language model (LLM) agents now write a large share of the code in some model-driven engineering (MDE) tools, under human direction and across many sessions. The domain knowledge that should survive this process is carried by markup documents whose role in the loop is, in practice, informal. We treat that apparatus, the harness, as the object of study: its markup documents are its instruction set. We reconstruct the harness of *jModel*, an MDE workbench of about 370k lines built by a three-executor loop (a human director, an architect agent, and an implementer agent), as an FTG+PM model grounded in repository evidence, lifting the classic manual-or-automatic typing of transformations to a three-executor typing. On top of this model we define knowledge debt as a conformance property: six recurring forms of knowledge loss become conformance gaps between the enacted process and its own type space. We then turn the model into an assessment instrument and apply it to a corpus of 233 authored markup documents, obtaining seven re-runnable indicators. Two are sharp: only about 4 to 5 percent of discovery reports are carried forward in writing to a downstream consumer, and a rotation invariant the persistent context store states for itself is violated by a factor of 25. The contribution is not the formalism, which is prior art, but the application of it to an LLM-agentic development harness and the reframing of knowledge fidelity as measurable conformance. We report the indicators with an honest account of what an n equals 1 self-study can and cannot show.

KEYWORDS

model-driven engineering, large language models, FTG+PM, process modelling, knowledge debt, technical debt, conformance, agentic development

1 INTRODUCTION

A growing fraction of software, including model-driven engineering (MDE) tooling, is now produced by large language model (LLM) agents working under human direction. In the system we study, the human author writes close to none of the code; the work is deciding what to build and maintaining the apparatus that lets agents build it reliably across sessions. That apparatus is a set of

markup documents that carry intent, context, and verdicts between the agents. These documents are not passive notes: they are the control surface of the process.

The difficulty is that this control surface is informal. Documents accumulate, drift from the code, and encode assumptions that live only in the director's head. The accidental complexity of writing code has been reduced, but the essential complexity of keeping knowledge faithful to the system has not [1]. We call the gap between the domain knowledge a document should incarnate and the knowledge actually internalized by its author or its consuming agents *knowledge debt*, by analogy with technical debt [2, 3] but distinct from it: knowledge debt is about what the artifacts fail to carry, not about the quality of the code.

This paper makes the harness itself the object of study and asks whether it can be formalized, made auditable, and assessed in terms of knowledge fidelity rather than only speed or volume. Our instrument is FTG+PM [4, 5], a formalism that types every artifact by a language and every step by a transformation. FTG+PM is prior art, and we claim no novelty in the formalism. The novelty is twofold: (i) applying it to a heavily LLM-assisted development harness, with the manual-or-automatic attribute lifted to a three-executor typing; and (ii) reframing knowledge debt as a conformance property of the resulting model, which makes it measurable.

Research questions.

- RQ1** Can a heavily LLM-assisted MDE development harness be reconstructed as an FTG+PM model grounded in repository evidence?
- RQ2** Does reframing knowledge debt as conformance gaps in that model yield measurable indicators of knowledge fidelity?
- RQ3** What do those indicators reveal about where domain knowledge thins out in the harness?

Contributions. (1) The harness-topology framing: markup documents as the typed instruction set of a three-executor loop. (2) An FTG+PM model of the *jModel* harness with executor typing, kept conformant to the metamodel. (3) Knowledge-debt-as-conformance: six classes of knowledge loss expressed as model conformance gaps. (4) An assessment instrument: seven re-runnable indicators applied

to a 233-document corpus, with two sharp results and an honest reading of the threats.

2 BACKGROUND

2.1 FTG+PM

The Formalism Transformation Graph and Process Model (FTG+PM) [4, 5] has two coupled parts. The FTG is the type level: a graph whose nodes are *languages* (document formalisms) and whose edges are *transformations* between them. The PM is the enactment: an activity diagram whose Activity nodes are each typed by a transformation and whose Object nodes are each typed by a language, connected by control flow and data flow, with the usual control nodes (Initial, Decision, Fork, Join, Final). The discipline it imposes for free is that every artifact in the process is typed by a formalism and every activity by a transformation. As its author notes, the value of the FTG is reasoning at the language level about chains of transformations over heterogeneous models; the activity-diagram part is, deliberately, ordinary. We rely on exactly this discipline, not on any extension of the formalism.

2.2 The LLM-agentic development loop

The harness we study has three executors. The *human director* authors intent, sets constraints, and gates the loop, accepting, rejecting, or redirecting. The *architect* (a conversational LLM) performs reasoning, design, and prompt generation. The *implementer* (a coding agent) edits the code base against prompt artifacts and emits build or test feedback. Persistent context (a constitution-like document read before every task), transient handoffs (briefs and prompts), and skills (lazily loaded competence descriptions) condition each step. The director writes essentially no code; the leverage is in the documents.

3 THE JJODEL HARNESS AS OBJECT OF STUDY

jJodel is a web-based MDE workbench (metamodelling, model editing, a transformation language and an expression language, Ecore and XMI import and export) of about 370k lines, a large share produced by the implementer agent under direction. Its harness is the set of markup documents that flow between the three executors.

The director plays two roles, and the model must keep them apart. *Authoring* intent is an Activity typed by a manual transformation; *gating* produces no artifact and is a Decision control node. Collapsing the two would erase the loop's only deterministic control point. The director functions as a weak oracle: acceptance is a human judgement, not an automated test.

The markup control surface falls into roles: persistent context (the constitution and agent memory), handoff and spec (task briefs, prompt artifacts), target and record (code, logs), verdict and review (discovery reports, audits), governance (conventions and policies), and domain and language (the language specifications that enter the loop). A single feature thread runs: the director states intent, the architect produces a design and a discovery query over the code base, a discovery report comes back, the director gates on a risk and effort trade-off, the implementer changes the code, the change

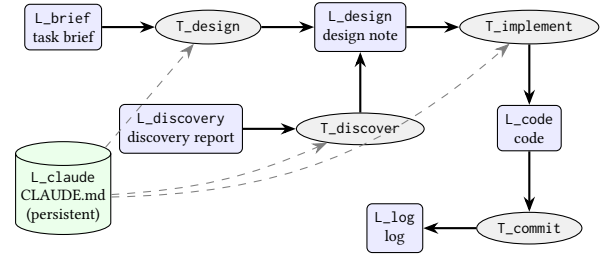


Figure 1: Core of the harness FTG. Languages (boxes) are produced and consumed by transformations (ellipses); the persistent context L_claude (cylinder) is read by many transformations. The full FTG has 11 languages and 13 transformations.

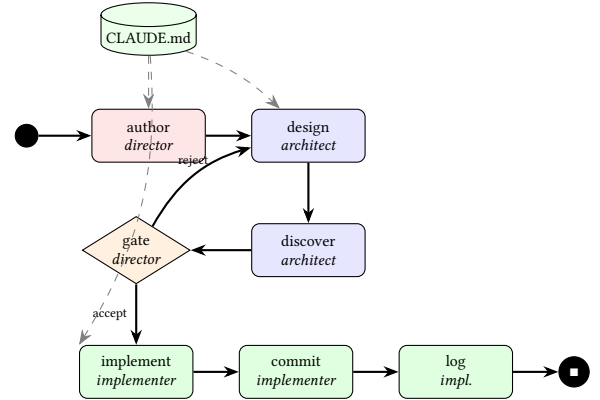


Figure 2: The enacted PM. Activities are coloured by executor (director, architect, implementer); the gate is a Decision with a reject back-edge. The persistent Object CLAUDE.md feeds several activities by data flow (dashed). Transient objects between activities are omitted for space.

is committed, and the log is appended. This thread is the running example for the model.

4 MODELLING THE HARNESS IN FTG+PM

We reconstructed the FTG and the PM from repository evidence (the document corpus and git history) and serialized them as a metamodel-conformant model. The current FTG has 11 languages and 13 transformations; the PM has 9 activities, 7 objects, 3 decisions, an initial and a final node. Figure 1 shows the core of the FTG and Figure 2 the enacted cycle.

Executor typing. Classic FTG+PM types a transformation as manual or automatic. The harness has three executors, so we keep the metamodel attribute *auto* (true for an agent step, false for a human or co-performed step) and record the specific executor as a leading tag in the transformation definition, drawn from director, architect, implementer, or collaborative. This stays conformant to the metamodel while preserving the three-agent identity the analysis

depends on. Where executor should become first-class, the meta-model can be extended with an `Executor` enumeration; until then the tag convention holds.

A correction the modelling surfaced. An early machine-generated version of the metamodel constrained a transformation to exactly one output language. This is wrong in general: a language can feed many transformations and a transformation can produce many languages (for example, one step emitting both a code model and a configuration model). The corrected cardinality is many-to-many, and the serialized model reflects it.

Validation and divergence. We validate each PM fragment against observed traces. Where the enacted process exceeds, skips, or violates the FTG type space, we record the divergence as a finding rather than editing the model into false conformance. The divergence is data, and in this study it is also the evidence for the assessment.

5 KNOWLEDGE DEBT AS CONFORMANCE

Under this model, knowledge debt surfaces as failure to conform. We distinguish six classes, each tied to the metamodel feature it violates:

- (1) a document that no `Language` can type: the FTG is incomplete (missing language);
- (2) a real step that no `Transformation` can type: a missing, often undocumented, transformation;
- (3) a persistent `Object` whose `Language` definition no longer matches the code it describes: a stale context store;
- (4) a `Transformation` whose outputs never appear as a written `Object`, or an edge that never fires: a dead or merely aspirational edge;
- (5) a `dataFlow` into an `Activity` from an `Object` written many iterations earlier: an activity reading stale context;
- (6) a transient handoff that silently carries assumptions absent from every `Language` definition: undocumented knowledge in transit.

Counting and characterizing these gaps is how we assess the harness. The debt register and the assessment are two readings of the same data.

6 ASSESSMENT

6.1 Instrument and corpus

The corpus is the authored markup of the repository at a fixed commit: 233 markdown files (excluding vendored and build copies), plus the serialized FTG+PM model and git history. Every indicator below is produced by a scripted pass over this corpus and is therefore re-runnable. The indicators generalize beyond `jJodel`: they are parameters of a harness, computable from its model and corpus (static) or from its execution logs (dynamic).

6.2 Two sharp results

Handoff incarnation. The harness produces discovery reports: queries over the code base whose answers should constrain the subsequent implementation. The corpus holds 97 dated discovery reports. Excluding the append-only log, which records every file

Table 1: Assessment indicators, with values for `jJodel` at the fixed commit. Static indicators are computable from the model and corpus; dynamic ones need execution logs.

Indicator	Value	Kind
Document typing coverage	≈60% (141/233)	static
Handoff incarnation rate	≈4–5% (4–5/97)	static
Persistent-context freshness	1 of 2 stores stale	static
Governance-rule conformance	1 invariant violated 25×	static
Retired-edge fraction	1 of 11 languages	static
Derived-doc drift	3–4% size of source spec	static
Type-authority presence	absent (.ecore)	static

touched, only 4 of them (5 if a codebase-overview analysis counts as a consumer) are referenced by any downstream design, spec, or implementation document. The discovery-to-implementation data flow is therefore largely not incarnated: the report is produced, read once by the director in session, and not carried forward as a cited object. This is class (6) of the lens, undocumented knowledge in transit, measured at about 4 to 5 percent.

Governance conformance. The persistent context store states a rotation invariant for the operational log: keep only the last 20 entries in the active file and move the rest to an archive. In fact the active log holds 514 dated entries and no archive file exists. A persistent `Object` asserts an invariant in its own `Language` definition that the process does not keep, a violation by a factor of 25. This is class (3) sharpened to a definition-versus-artifact conformance failure.

6.3 The other indicators

Document typing coverage is about 60 percent: three languages (discovery, handover, design) type 141 of 233 documents, while the rest sit in roles the 11-language FTG lacks or under-specifies (verdict and audit records, agent memory, a twin constitution), instances of class (1). Persistent-context freshness: of two constitution-like stores, one is fresh (updated the day of measurement) and one, a near-duplicate created for a second agent, has not been touched in 13 days while its twin changed, class (3). Retired edge: the handover language was produced in 24 documents through February and never since, so its production transformation no longer fires, class (4). Derived-document drift: user-facing reference pages are 3 to 4 percent the size of the specifications they mirror, a stale derived document. Type-authority presence: the designated metamodel authority (the .ecore) is absent from the corpus, which weakens the falsifiability of the model and is itself a finding.

From debt to parameters. The seven indicators instantiate a small parameter model for any harness: knowledge fidelity (coverage, incarnation, freshness, conformance, drift, edge liveness, provenance), process structure (type-space size, loop depth, executor distribution), governance (strictness calibration, gate effectiveness),

and runtime non-determinism (skill-selection variance, trace coherence). The first family is what this model measures as conformance; the last is the dynamic extension we discuss next.

7 DISCUSSION AND THREATS TO VALIDITY

The study is n equals 1: one harness and one director, who is also the analyst, assisted by the same family of agents that run the harness. Generality is not claimed; the contribution is an instrument and an existence proof that the indicators are computable and informative. Several indicators depend on classification choices a reviewer can contest, for example what counts as a downstream consumer or as cleanly typed; we report the rules so the counts can be reproduced and challenged. The indicators measure incarnation and conformance, not whether the resulting software is correct; the hypothesized link from knowledge fidelity to outcome quality is not demonstrated here.

A dynamic dimension remains open. Skill selection in the harness is non-deterministic: the agent matches a task to a skill description by inference, not by lexical match, evidence for which comes from interpretability work showing concept-level features that fire across languages and modalities [7]. Some participants hold that the underlying mechanism is still probabilistic and that the two accounts may be observationally close; we present both readings. Either way, logging which skills fire per run would let process mining [8] measure trace coherence and flag runs that enter a risky zone, turning the static instrument into a runtime monitor.

8 RELATED WORK

MDE and LLMs. Most work applies foundation models to modelling artifacts: generation of metamodels, transformations, and concrete syntax, and prompt engineering for reliable outputs. We invert the direction: rather than using AI to produce models, we model the AI-assisted process itself.

Process and workflow modelling. Strict procedural workflows gave way to declarative process models that assert properties rather than order [6], and to conformance checking and process mining over logs [8]. Our PM is deterministic at the level the director drives it but internally non-deterministic per activity; the natural assessment is property checking over execution traces, which connects directly to that line.

Technical versus knowledge debt. Technical debt is a property of the code and its structure [2, 3]. Knowledge debt, as we use it, is a property of the documents and their conformance to a model of the process. The two are orthogonal: a code base can be clean yet leak knowledge at every handoff.

Harness and agentic-workflow engineering. Practitioner accounts describe constitutions, discovery and implementation prompts, and skills, but informally. To our knowledge none casts agentic-development knowledge loss as FTG+PM conformance gaps, which is the delta of this paper.

9 CONCLUSION AND FUTURE WORK

We reconstructed an LLM-assisted MDE development harness as an FTG+PM model grounded in repository evidence (RQ1), reframed knowledge debt as conformance gaps in that model and showed the reframing yields measurable indicators (RQ2), and reported

where knowledge thins out: at the discovery handoff, at a self-stated governance invariant, and across roles the type space does not cover (RQ3). The formalism is prior art; the application and the assessment lens are not. Looking forward, the validated PM can be turned into a prescriptive harness, with each language as a template and each transformation as a contract the agents follow, and the static instrument can be extended into a runtime coherence monitor over skill traces. The model is a falsifiable hypothesis about the harness, and when practice and the model conflict, practice wins as evidence and the model is what gets revised.

REFERENCES

- [1] Frederick P. Brooks. 1987. *No Silver Bullet: Essence and Accidents of Software Engineering*. IEEE Computer. Essence versus accidental complexity; verify venue.
- [2] Ward Cunningham. 1992. The WyCash Portfolio Management System. *ACM SIGPLAN OOPS Messenger* 4, 2 (1992), 29–30.
- [3] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. 2012. Technical Debt: From Metaphor to Theory and Practice. *IEEE Software* 29, 6 (2012), 18–21.
- [4] Levi Lúcio, Sadaf Mustafiz, Joachim Denil, Hans Vangheluwe, and Maris Jukss. 2013. FTG+PM: An Integrated Framework for Investigating Model Transformation Chains. In *SDL 2013: Model-Driven Dependability Engineering (Lecture Notes in Computer Science, Vol. 7916)*. Springer, 182–202.
- [5] Sadaf Mustafiz, Joachim Denil, Levi Lúcio, and Hans Vangheluwe. 2020. The FTG+PM Framework for Multi-Paradigm Modelling. In *Multi-Paradigm Modelling Approaches for Cyber-Physical Systems*. Springer. Book chapter; verify exact title, editors and pages.
- [6] Maja Pesic and Wil M. P. van der Aalst. 2006. A Declarative Approach for Flexible Business Processes Management. In *Business Process Management Workshops (Lecture Notes in Computer Science, Vol. 4103)*. Springer, 169–180.
- [7] Adly Templeton, Tom Conerly, Jonathan Marcus, et al. 2024. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. Transformer Circuits Thread. Anthropic. Verify author list and URL.
- [8] Wil M. P. van der Aalst. 2016. *Process Mining: Data Science in Action* (2 ed.). Springer.